Linear classifiers and explicit feature maps

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

A linear classifier separates classes with a hyperplane. Many algorithms learn one — nearest centroid, Gaussian discriminant analysis, least squares, the perceptron, logistic regression, support vector machines — and they differ only in which hyperplane they choose. This note develops their geometry and their training, explains why a hyperplane is often not enough, and shows the oldest way around it: map the inputs non-linearly and classify linearly in the new space. Learning that map instead of fixing it is what a neural network does, the subject of the rest of week 4.

1 What makes a classifier linear

A binary classifier is *linear* if its decision function has the form

$$f(\mathbf{x}) = \operatorname{sign}\big(g(\mathbf{x})\big), \qquad g(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b,$$

with labels $t \in \{-1, +1\}$, weight vector $\mathbf{w} \in \mathbb{R}^d$ and bias b. For m classes, $f(\mathbf{x}) = \arg\max_c (W\mathbf{x} + \mathbf{b})_c$ with a weight matrix $W \in \mathbb{R}^{m\times d}$ and a bias vector; for $m = 2$ the two forms agree, with $\mathbf{w} = W_{2:} - W_{1:}$ and $b = b_2 - b_1$ (Note 6, Section 1).

Geometry. The decision boundary $H = \{\mathbf{x} : \mathbf{w}^{\mathsf{T}}\mathbf{x} + b = 0\}$ is a hyperplane — a line in two dimensions, a plane in three. The vector $\mathbf{w}$ is normal to it, and the signed distance of a point to H is

$$r(\mathbf{x}) = \frac{\mathbf{w}^{\mathsf{T}}\mathbf{x} + b}{\|\mathbf{w}\|}, \tag{1}$$

so $g(\mathbf{x})$ is a distance measured in units of $1/\|\mathbf{w}\|$. The two decision regions are the half-spaces on either side. In the multiclass case each region $\{\mathbf{x} : g_c(\mathbf{x}) > g_{c'}(\mathbf{x})\ \forall c' \neq c\}$ is an intersection of half-spaces, hence convex.

Linear separability. A dataset is *linearly separable* if some hyperplane classifies all of it correctly: there exist $\mathbf{w}, b$ with $t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b) > 0$ for every i. Many datasets are not. The classic example is XOR — the points $(0,0)$ and $(1,1)$ in one class, $(0,1)$ and $(1,0)$ in the other — which no line separates; its discussion in Minsky and Papert's book of 1969 slowed research on neural networks for a decade [7].

2 Many roads to a hyperplane

The following algorithms all output a linear decision function; for the same data they choose different hyperplanes, which generalise differently. Figure 1 shows five of them on one dataset.

Nearest centroid. Store the class means $\boldsymbol{\mu}_c$ and assign $\mathbf{x}$ to the nearest. Since $\|\mathbf{x}-\boldsymbol{\mu}_c\|^2 = \|\mathbf{x}\|^2 - 2\boldsymbol{\mu}_c^{\mathsf{T}}\mathbf{x} + \|\boldsymbol{\mu}_c\|^2$ and $\|\mathbf{x}\|^2$ is common to all classes, the rule is linear with $\mathbf{w}_c = 2\boldsymbol{\mu}_c$ and $b_c = -\|\boldsymbol{\mu}_c\|^2$; for two classes the boundary is the perpendicular bisector of the segment between the means.

Gaussian discriminant analysis. With Gaussian classes sharing one covariance, the Bayes classifier is linear, $\mathbf{w} = \Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$ (Note 6, Section 3). It accounts for the shape of the classes, which nearest centroid ignores. Fisher's linear discriminant, derived differently — the projection that best separates the class means relative to the spread within classes — gives the same direction.

Least squares on ± 1 targets. Nothing prevents fitting linear regression (Note 8) to the labels t_i and classifying by the sign of the prediction. But squared error penalises points that are correctly classified *with a large margin*, since their prediction is far from ± 1. A distant cluster of easy examples therefore pulls the boundary towards itself; in Figure 1 the twelve far-away amber points tilt the least-squares line and triple its training error (9.8%) compared with logistic regression (3.3%).

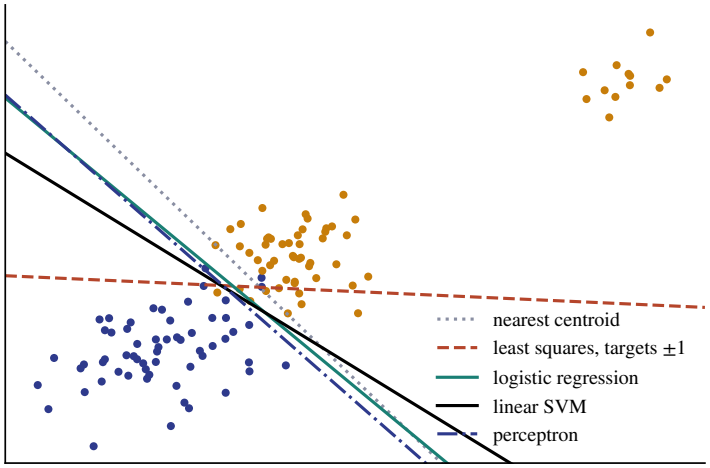


Figure 1. Five linear classifiers on the same 122 points. All boundaries are straight lines, but they differ. Training errors: nearest centroid 4.9%, least squares 9.8%, logistic regression, linear SVM and perceptron 3.3%. The least-squares line is pulled by the distant amber cluster in the top right.

Minimising the error directly. The training error rate is piecewise constant in $(\mathbf{w}, b)$: its gradient is zero almost everywhere and gradient descent (Note 7) cannot move. The remaining algorithms minimise surrogates (Note 4, Section 2) that do have useful gradients.

3 The perceptron

Rosenblatt's perceptron [11], built in hardware as the Mark I in the late 1950s, was one of the first learning machines and the first artificial neuron trained from data. Starting from $\mathbf{w} = \mathbf{0}$, $b = 0$, it visits the examples in random order and corrects mistakes:

repeat
  for each $(\mathbf{x}_i, t_i)$ in random order **do**
    if $t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b) \le 0$ **then** $\mathbf{w} \leftarrow \mathbf{w} + t_i\mathbf{x}_i, \quad b \leftarrow b + t_i$
    end if
  end for
until no mistakes in a full pass (or a maximum number of passes)

This is stochastic gradient descent with step $\eta = 1$ on the *perceptron criterion*

$$J(\mathbf{w}, b) = \sum_{i \text{ misclassified}} -t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b),$$

which measures how far misclassified points lie on the wrong side. If the data are separable with margin γ (some unit-norm $\mathbf{w}^*$ has $t_i \mathbf{w}^{*\mathsf{T}} \mathbf{x}_i \geq \gamma$) and $\|\mathbf{x}_i\| \leq R$, the perceptron makes at most $(R/\gamma)^2$ mistakes before separating them [8] — a bound independent of n and d. If the data are not separable it never stops: on the data of Figure 1 it made 606 corrections in 100 passes. It also stops at *any* separating hyperplane, however close to the data, which motivates the margin of Section 5.

4 Logistic regression

Despite its name, logistic regression [5] is a classifier. With $t \in \{0, 1\}$ it models the class probability directly,

$$P(t = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^{\mathsf{T}} \mathbf{x} + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}},$$

the logistic sigmoid, a smooth version of a step function that maps the score to $(0, 1)$. It is trained by minimising the cross-entropy (Note 4, Section 2), the negative log-likelihood of the labels. With $y_i = \sigma(\mathbf{w}^{\mathsf{T}} \tilde{\mathbf{x}}_i)$ and the bias absorbed as in Note 8,

$$J(\mathbf{w}) = -\sum_{i=1}^{n} \big[t_i \log y_i + (1 - t_i) \log(1 - y_i) \big], \qquad \nabla J = \sum_{i=1}^{n} (y_i - t_i)\, \tilde{\mathbf{x}}_i .$$

The gradient has the same form as for least squares — prediction minus target, times input — because $\sigma' = \sigma(1 - \sigma)$ cancels a factor. The objective is convex, but $\nabla J = \mathbf{0}$ has no closed-form solution; it is minimised by gradient descent, by mini-batch SGD for large data, or by Newton's method (Note 7), which converges in a few iterations. Two practical points: if the data are separable, $\|\mathbf{w}\|$ grows without bound because sharper sigmoids keep lowering the loss, so a ridge penalty (Note 8) is added; and with m classes the sigmoid becomes the *softmax*, $P(t = c \mid \mathbf{x}) = e^{g_c(\mathbf{x})} / \sum_{c'} e^{g_{c'}(\mathbf{x})}$, trained with the categorical cross-entropy.

Since $\sigma(z) > \frac{1}{2}$ exactly when $z > 0$, predicting the most probable class is still $\operatorname{sign}(\mathbf{w}^{\mathsf{T}} \mathbf{x} + b)$: the boundary is a hyperplane. What logistic regression adds is a calibrated probability, and a loss that, unlike squared error, does not punish confident correct predictions. In neural terms it is a single neuron with a sigmoid activation.

> **Modern connection — every neural classifier ends in logistic regression**
>
> The last layer of an image classifier or a language model is a linear map followed by a softmax: multiclass logistic regression on features computed by the layers below. Conversely, training only a logistic regression on the frozen features of a pre-trained network — a *linear probe* — is the standard way to measure what those features contain [1]; CLIP, for instance, was evaluated with linear probes on 27 datasets [9]. Everything in this note therefore applies on top of the most modern representations.

5 Support vector machines

Among all separating hyperplanes, the *maximum-margin* one stays as far as possible from the closest training points. The *functional margin* of an example is $t_i g(\mathbf{x}_i)$ — positive if it is correctly classified, large if confidently so — and its *geometric margin* is $t_i g(\mathbf{x}_i) / \|\mathbf{w}\|$, its signed distance to the boundary by (1). Since rescaling $(\mathbf{w}, b)$ does not move the hyperplane, we may fix the smallest functional margin to one; maximising the geometric margin $1/\|\mathbf{w}\|$ then becomes the *hard-margin SVM*:

$$\min_{\mathbf{w}, b} \tfrac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad t_i(\mathbf{w}^{\mathsf{T}} \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n,$$

a quadratic programme with linear constraints [4]. For data that are not separable, *slack* variables $\xi_i \geq 0$ allow violations at a price: minimise $\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i$ subject to $t_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) \geq 1 - \xi_i$. Eliminating the slacks gives an unconstrained form,

$$\min_{\mathbf{w},b} \sum_{i=1}^{n} \max\left(0,\, 1 - t_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b)\right) + \frac{1}{2C}\|\mathbf{w}\|^2 ,$$

the hinge loss of Note 4 plus a ridge penalty. The hyperparameter C trades margin against training errors; $C \to \infty$ recovers the hard margin.

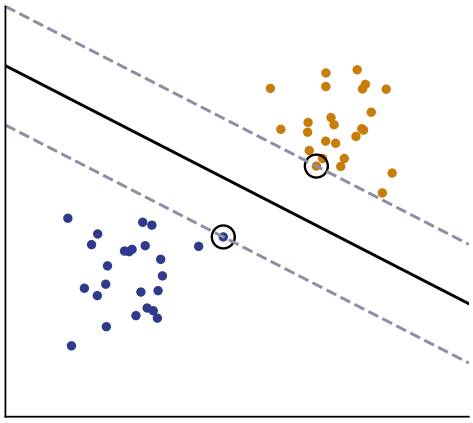


Figure 2. A maximum-margin classifier. The dashed lines are $g(\mathbf{x}) = \pm 1$; the margin between them has width $2/\|\mathbf{w}\|$. Only the two circled points — the support vectors — determine the solution: moving any other point, without crossing the dashed lines, changes nothing.

Support vectors. The solution has the form $\mathbf{w} = \sum_i \alpha_i t_i \mathbf{x}_i$ with $\alpha_i \geq 0$, and only points on or inside the margin have $\alpha_i > 0$ (Figure 2). These *support vectors* are usually a small fraction of the data. Because $\mathbf{w}$ is a combination of training inputs, the problem can be rewritten in terms of the α_i and inner products $\mathbf{x}_i^\mathsf{T}\mathbf{x}_j$ alone — the dual formulation, which opens the way to the kernel trick of Section 6. Soft-margin SVMs, linear or kernelised, were among the most widely used classifiers of the 2000s: good accuracy, a convex problem and few hyperparameters.

More than two classes. Binary classifiers extend in two standard ways: *one-against-all*, with m classifiers each separating one class from the rest and the highest score winning; and *one-against-one*, with $m(m-1)/2$ pairwise classifiers and a vote. Taking the highest score, rather than the raw yes/no answers, avoids the ambiguous regions where several or no classifiers claim a point.

6 Beyond linear: explicit feature maps

Whatever the algorithm, a linear classifier draws a hyperplane, and its capacity is limited. The oldest remedy keeps the linear algorithm and changes the space: apply a fixed non-linear *feature map* $\phi : \mathbb{R}^d \to \mathbb{R}^D$ and learn a linear classifier on $\phi(\mathbf{x})$,

$$f(\mathbf{x}) = \operatorname{sign}\left(\mathbf{w}^\mathsf{T}\phi(\mathbf{x}) + b\right).$$

The boundary is a hyperplane in the new space and a curved surface in the original one. For two inputs, the quadratic map $\phi(x_1, x_2) = (x_1^2, x_2^2, x_1x_2, x_1, x_2)$ makes $\mathbf{w}^\mathsf{T}\phi(\mathbf{x}) + b$ an arbitrary second-degree polynomial, so the boundary can be any conic. In Figure 3 a class that surrounds the other defeats logistic regression on $\mathbf{x}$ (49% training error, chance level), while the same algorithm on $\phi(\mathbf{x})$ separates the classes perfectly with a circle.

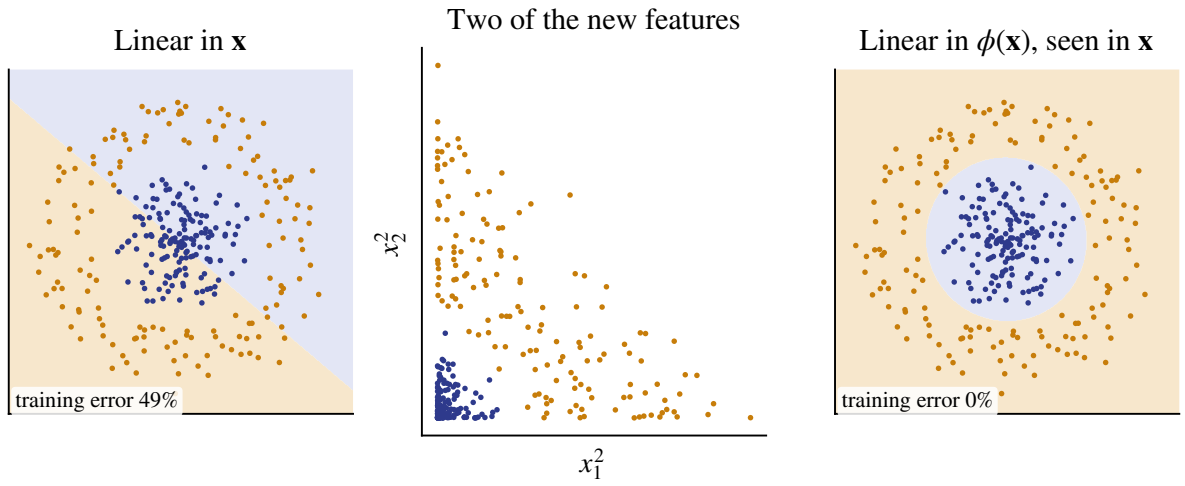


Figure 3. Left: logistic regression on the raw inputs. Centre: two of the five quadratic features, in which the classes are separated by a straight line. Right: logistic regression on all five quadratic features, with the boundary drawn in the original coordinates.

The cost of explicit maps. All monomials of degree at most p in d variables number $\binom{d+p}{p}$: 21 for $d = 2, p = 2$, but 176 851 for $d = 100, p = 3$. The capacity grows accordingly, and with it the need for regularisation and data (Note 4) — the curse of dimensionality in a new form (Note 3). There are three ways to obtain a non-linear feature map:

1. **Choose it explicitly**, a priori, as above: polynomials, physically motivated variables, or random non-linear features — the random ReLU features of Note 4, Section 8, are an example, and random Fourier features approximate kernel machines at large scale [10].
2. **Choose it implicitly** through a kernel $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^{\mathsf{T}}\phi(\mathbf{x}')$: algorithms that need only inner products, like the dual SVM, can work in very high or even infinite-dimensional feature spaces without ever computing ϕ (kernel SVM, kernel logistic regression).
3. **Learn it**: let ϕ have its own parameters and fit them together with $\mathbf{w}$. A multilayer perceptron does exactly this, $f(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}a(V\mathbf{x} + \mathbf{c}) + b$, with a learned matrix V and an element-wise non-linearity a: a linear classifier on features that are themselves trained by gradient descent (Note 7). This is where week 4 continues.

In physics — feature engineering, the physicist's way

The invariant mass of a pair of photons, $m_{\gamma\gamma}^2 = 2E_1E_2(1 - \cos\theta_{12})$, is a non-linear feature map of the measured energies and angles. In it, a Higgs boson decaying to two photons is a narrow peak, and a simple window separates much of the signal — something no hyperplane in the raw measurements could do. Physics has always supplied such features. When deep networks were given only low-level detector quantities in benchmark searches for new particles, they matched or exceeded the performance obtained with the best hand-engineered variables, having learned equivalent features themselves [2].

Summary A linear classifier predicts $\text{sign}(\mathbf{w}^{\mathsf{T}}\mathbf{x} + b)$; its boundary is a hyperplane normal to $\mathbf{w}$, and $g(\mathbf{x})/\|\mathbf{w}\|$ is a signed distance. Nearest centroid, LDA, least squares, the perceptron, logistic regression and the SVM all learn hyperplanes, chosen by different criteria: least squares is pulled by distant points, the perceptron stops at any separator, logistic regression gives cali-

brated probabilities with a convex loss, and the SVM maximises the margin and depends only on its support vectors. A non-linear feature map makes linear methods non-linear: chosen explicitly, implicitly through kernels, or learned, as in a neural network.

Exercises

E1. Show that $\mathbf{w}$ is normal to the hyperplane $\mathbf{w}^{\mathsf{T}}\mathbf{x}+b=0$ and derive the distance formula (1). What is the distance from the origin to the hyperplane?

E2. Show that the XOR points are not linearly separable, and find a single extra feature $\phi_3(\mathbf{x})$ such that they become separable in (x_1, x_2, ϕ_3).

E3. Derive ∇J for logistic regression using $\sigma' = \sigma(1-\sigma)$, and the Hessian $\sum_i y_i(1-y_i)\,\tilde{\mathbf{x}}_i\tilde{\mathbf{x}}_i^{\mathsf{T}}$. Conclude that J is convex. Why does $\|\mathbf{w}\|$ diverge on separable data?

E4. Implement the perceptron and run it on separable data with margin γ and radius R of your choice. Compare the number of mistakes with the bound $(R/\gamma)^2$.

E5. Explain why the solution of the soft-margin SVM does not change when a correctly classified point outside the margin is moved, as long as it stays outside the margin. Which points of Figure 1 would a linear SVM ignore?

E6. How many features does the full polynomial map of degree 3 produce for 28×28 images? Estimate the memory needed to store them for 60 000 images in 32-bit floats, and suggest an alternative.

Further reading. Bishop [3], chapter 4 and section 7.1; Hastie, Tibshirani and Friedman [6], sections 4.4–4.5 and 12.2–12.3; for the history of the perceptron, Rosenblatt [11].

References

[1] G. Alain and Y. Bengio, Understanding intermediate layers using linear classifier probes, *ICLR Workshop* (2017).

[2] P. Baldi, P. Sadowski and D. Whiteson, Searching for exotic particles in high-energy physics with deep learning, *Nat. Commun.* 5, 4308 (2014).

[3] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[4] C. Cortes and V. Vapnik, Support-vector networks, *Mach. Learn.* 20, 273–297 (1995).

[5] D. R. Cox, The regression analysis of binary sequences, *J. R. Stat. Soc. B* 20, 215–242 (1958).

[6] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer (2009).

[7] M. Minsky and S. Papert, *Perceptrons*, MIT Press (1969).

[8] A. B. J. Novikoff, On convergence proofs on perceptrons, *Proc. Symposium on the Mathematical Theory of Automata* 12, 615–622 (1962).

[9] A. Radford *et al.*, Learning transferable visual models from natural language supervision, *ICML* (2021).

[10] A. Rahimi and B. Recht, Random features for large-scale kernel machines, *NeurIPS* (2007).

[11] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain, *Psychol. Rev.* 65, 386–408 (1958).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.